Software Entwurf 4
Übung 2

Sommersemester 2026

HSD FH-HAGENBERG

(1) Name: Simon Offenberger
Mat.Nr.: S2410306027

(2) Name: Jakob Renner
Mat.Nr.: S2410306009
Übungsgruppe: 1
Aufwand in Mh: 10

Abgabetermin: KW 15
Punkte: _____
korrigiert: _____

Beispiel 1 (10 Punkte) Parallelisierung: Ihr Logistik-Unternehmen verfügt über 8 LKWs, die in acht verschiedenen Niederlassungen stationiert sind. Für einen Hilfsgütertransport wurden ihnen an allen Standorten eine Vielzahl an Gütern abgegeben - viel mehr, als Sie auf einem LKW transportieren können. Sie stehen vor dem Problem, nun jene Güter auszuwählen, die bei der begrenzten Ladekapazität ihres LKWs den maximalen Nutzen für das Katastrophengebiet haben (maximaler Wert bei geringstem Gewicht).

Entwickeln Sie für ihre Zentrale eine Software, die diese Optimierung für die 8 LKW durchführt. Jedes Gut hat ein Gewicht und einen Nutzen (bzw. Wert). Ihr LKW hat eine maximale Nutzlast. Um herauszufinden, welche Güter Sie mitnehmen, müssen Sie alle Güterkombinationen ausprobieren - für jeden LKW getrennt.

Lassen Sie sich vom vorhandenen Programm `Optimierung.cpp` inspirieren und machen Sie sich mit der Funktionsweise des Algorithmus vertraut.

a) Entwickeln Sie eine sequenzielle Lösung, wo Sie alle 8 LKW-Ladungen hintereinander berechnen. Messen Sie die Zeitdauer und kommentieren Sie diese.

b) Führen Sie zur Beschleunigung der Berechnung eine Parallelisierung durch, in dem Sie jeden einzelnen LKW jeweils in einem eigenen Thread (insges. 8 Workerthreads) rechnen. Welchen Speedup können Sie auf ihrer Hardware erreichen? Dokumentieren Sie zusammen mit der verwendeten Hardware die erzielten Ergebnisse.

Anmerkungen:

- Legen Sie für jeden LKW eine eigene ASCII-Datei an, die Sie jeweils einlesen. Die Datei speichert die LKW-Nutzlast sowie Gewicht & Nutzen alle Güter. (Anm.: Nehmen Sie - vereinfachend - für jeden LKW die gleiche Datei.)
- Transportieren Sie rund 40 - 50 Güter pro LKW. Sollte ihr Rechner zu langsam sein, reduzieren Sie individuell die Anzahl, damit Sie akzeptable Laufzeiten bekommen.
- Legen Sie die Nutzlast so fest, dass Sie rund 50% der Güter transportieren können.
- Sie haben nur eine Fahrt, dürfen den LKW nicht überladen, und keine Güter zwischen den Standorten tauschen.

Hinweise zur Ausarbeitung:

- Führen Sie die Geschwindigkeitstests jeweils mit der **Release-Version** durch.
- Übersetzen Sie ihr paralleles Programm mit den **Multithreaded Bibilotheken** (Project Properties: C/C++ -> Code Generation -> Runtime Library: Multi-threaded DLL).
- Die **Terminierung von Threads** können Sie mit den API-Calls `WaitForSingleObject()` bzw. `WaitForMultipleObjects()` abwarten (siehe dazu die MSDN-Hilfe).
- Verwenden Sie die **Threadklasse** aus der Übung!

Beispiel 2 (14 Punkte) Admin-Tool: Um sich mit der Win32/64-Schnittstelle besser vertraut zu machen, implementieren Sie ein Administrationswerkzeug, das über folgende Funktionen verfügt:

1 Applikationen starten: Via Menüauswahl im Testtreiber können zumindest der *Editor* (`notepad.exe`) und der *Taschenrechner* (`calc.exe`) gestartet werden - auch mehrmals.

2 Prozesse auflisten: Alle Prozesse im System werden aufgelistet (sofern die Zugriffrechte existieren). Die Formatierung ist optional (`setw()`, `setfill()`, `left`, etc), zB:

```
Process        PID      BasePriority   Thread Count
-------------------------------------------------------
calc.exe       123      normal             1
notepad.exe    233      high               1
Mcshield.exe   215      low               18
calc.exe       317      low                1
Acrobat.exe    127      normal             4
...
```

3 Bestimmte Prozesse terminieren: Die über die PID angegebene Applikation wird terminiert, sofern die Rechte bestehen.

4 Systeminformation anzeigen: Hier wird eine Information über das aktuelle System ausgegeben (Hardware und Software). Sie dürfen sich hier auf die Intel-Architektur beschränken. Geben Sie also *ProcessorLevel* (Family) und *ProcessorRevision* (Model bzw. Stepping) nur aus, wenn es sich um Intel-CPUs handelt, ansonsten „unknown". Bei der Software beschränken Sie sich auf den Computernamen und die Betriebssystemversion, die ihre Applikation voraussetzt, wie auch ob Server-OS oder nicht. Eine Ausgabe kann etwa wie folgt aussehen:

```
System Information <HARDWARE>:
------------------------------
Processor Type                    : 586
Architecture                      : Intel CPU or compatibles
Processor Level (Family)          : 6
Revision                          : 0x2a07
Number of (logical) Processors    : 8

Features:
     Physical Address Extension not supported.
     Virtualization enabled.

System Information <SOFTWARE>:
------------------------------
Computer Name:   Zeus
OS Version   :   Windows 7 or greater
Server       :   No Server Version
```

Zur Realisierung:

Analyse: Analysieren Sie zuerst die Anforderungen und berücksichtigen Sie dabei, wo und wie ihnen die unten angeführten API-Calls helfen.

Design: Diskutieren Sie ihr Design und legen Sie die Datenstrukturen und Schnittstellen fest (Klassen & Methoden inkl. Parameter).

Implementierung: Implementieren Sie ihre Lösung mit Klassen (OOP).

Test: Erstellen Sie weiters einen Testtreiber, der diese Funktionen über ein einfaches Menü anbietet.

Dokumentation: Verfassen Sie eine Funktionsbeschreibung in LaTeX. Konzentrieren Sie sich dabei auf das Wesentliche.

Hinweise: Benützen Sie die Windows-Hilfe (Dokumentation zur Windows API), um sich mit den API-Calls und den Strukturen vertraut zu machen. Beginnen Sie bei folgenden API-Calls:

`GetCurrentProcess()`, `GetSystemInfo()`, `GetComputerName()`, `GetVersionEx()` bzw. die Version Helper Functions, `Process32First()`, `Process32Next()` bzw. Process Walking, `OpenProcess()`, `Set/GetPriorityClass()`, `TerminateProcess()`

Lassen Sie sich auch von dem jeweils angegebenen Beispiel-Code in der Hilfe inspirieren (etwa für die Prozessliste -> Bsp zu *Process Walking*), wo Sie die richtige Verwendung sehen.

Testen Sie ihr Programm ausführlich und geben Sie diese Tests auch ab!

Hinweise zur Ausarbeitung dieser Übung:

Lösen Sie diese Übung im Teamwork (2 Personen) und geben Sie eine gemeinsame Ausarbeitung ab. Setzen Sie dabei Bedarf an den entscheidenden Stellen **Pair Programming** ein, einer Technik aus dem Software Engineering Konzept *Extreme Programming* (XP von *Kent Beck*) und Scrum. Dabei sitzt das Teammitglied mit der geringeren Erfahrung am PC und codiert, während das zweite Teammitglied unterstützend und erklärend einwirkt und auch prüft, ob der geschriebene Code funktionieren kann (permanentes Review).

Sie können nach eigenem Ermessen im konventionellen Software Lifecycle arbeiten, oder weiter im XP- bzw. Scrum-Ansatz vorgehen.

- In XP/Scrum schreiben Sie **zuerst den Testtreiber** und werden sich dadurch bewusst, *welche Funktionalität* Sie benötigen und *in welcher Weise* Sie diese anwenden wollen.

- Danach beginnt das Design mit dem Fokus auf **Einfachheit** (*„the simplest thing that could possibly work“*).

- Die Implementierung wird in möglichst **kurzen Iterationen** immer ins Gesamtsystem integriert, um fehlende Übereinstimmungen möglichst früh zu erkennen. Sie können damit keine Funktionalitäten vergessen, weil Sie spätestens der nächste Testdurchlauf darauf hinweist.

- Da Testen zu hoher Produktstabilität führt, wird zwischendurch **immer wieder getestet** (*unit testing* durch SW-Entwickler und *functional testing* durch Kunden).

Allgemeine Hinweise: *Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine saubere Strukturierung und auf eine sorgfältige Ausarbeitung! Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den Testoutput mit ab!*
Erstellen Sie eine Funktionsbeschreibung in LaTeX und dokumentieren Sie, ***wer im Team welche Bereiche ausgearbeitet hat*** *- pro Beispiel!*